1. Fitur vertikal yang dibuat

Food Poison

Booking Food

Booking Form

Booking Model

Local Repository

SharedPreferences

Dengan begitu reviewer bisa melihat bahwa fitur benar-benar berjalan dari **Frontend sampai Local Storage**.

---

2. Posisi code Local Data & Persistence

Struktur `lib` yang saya sarankan:

```
lib/
├── main.dart
├── app.dart
│
├── models/
│   └── booking_model.dart
│
├── repositories/
│   └── booking_local_repository.dart
│
├── providers/
│   └── booking_provider.dart
│
├── screens/
```

```
│   └── booking_screen.dart
│
└── widgets/
    └── booking_form.dart
```

Yang ditunjukkan ke reviewer

Data Model

lib/models/booking_model.dart

Local Data / Storage

lib/repositories/booking_local_repository.dart

State Management

lib/providers/booking_provider.dart

Frontend

lib/screens/booking_screen.dart
lib/widgets/booking_form.dart

Jadi kalau reviewer bertanya:

> "Local persistence-nya ada di mana?"

Jawab:

> **"Local persistence terdapat di `lib/repositories/booking_local_repository.dart`, menggunakan `SharedPreferences`. Data yang disimpan menggunakan model `BookingModel`, sedangkan UI menggunakan `BookingScreen` dan state management menggunakan `BookingProvider`."**

---

3. Tambahkan dependency

Di `pubspec.yaml`:

```
dependencies:
  flutter:
```

```
  sdk: flutter
 flutter_riverpod: ^2.6.1
 shared_preferences: ^2.5.3
```

Kemudian:

```
flutter pub get
```

---

4. Data Model

Buat:

lib/models/booking_model.dart

```
class BookingModel {
 final String id;
 final String foodId;
 final String foodName;
 final String customerName;
 final String phone;
 final int quantity;

 BookingModel({
  required this.id,
  required this.foodId,
  required this.foodName,
  required this.customerName,
  required this.phone,
  required this.quantity,
 });

 Map<String, dynamic> toJson() {
  return {
   'id': id,
   'foodId': foodId,
   'foodName': foodName,
   'customerName': customerName,
   'phone': phone,
   'quantity': quantity,
  };
 }

 factory BookingModel.fromJson(Map<String, dynamic> json) {
```

```
    return BookingModel(
      id: json['id'] ?? '',
      foodId: json['foodId'] ?? '',
      foodName: json['foodName'] ?? '',
      customerName: json['customerName'] ?? '',
      phone: json['phone'] ?? '',
      quantity: json['quantity'] ?? 0,
    );
  }
}
```

Model ini menjelaskan **data apa yang disimpan secara lokal**.

---

5. Local Repository

Buat:

lib/repositories/booking_local_repository.dart

```
import 'dart:convert';

import 'package:shared_preferences/shared_preferences.dart';

import '../models/booking_model.dart';

class BookingLocalRepository {
  static const String _storageKey = 'food_poison_bookings';

  Future<List<BookingModel>> getBookings() async {
    final prefs = await SharedPreferences.getInstance();

    final data = prefs.getString(_storageKey);

    if (data == null) {
      return [];
    }

    final List<dynamic> jsonList = jsonDecode(data);

    return jsonList
        .map(
          (item) => BookingModel.fromJson(
```

```
          Map<String, dynamic>.from(item),
        ),
      )
      .toList();
}

Future<void> createBooking(BookingModel booking) async {
  final bookings = await getBookings();

  bookings.add(booking);

  await _save(bookings);
}

Future<void> updateBooking(BookingModel booking) async {
  final bookings = await getBookings();

  final index = bookings.indexWhere(
    (item) => item.id == booking.id,
  );

  if (index == -1) {
    return;
  }

  bookings[index] = booking;

  await _save(bookings);
}

Future<void> deleteBooking(String id) async {
  final bookings = await getBookings();

  bookings.removeWhere(
    (item) => item.id == id,
  );

  await _save(bookings);
}

Future<void> _save(
  List<BookingModel> bookings,
) async {
  final prefs = await SharedPreferences.getInstance();
```

```
    final jsonList = bookings
        .map((booking) => booking.toJson())
        .toList();

    await prefs.setString(
      _storageKey,
      jsonEncode(jsonList),
    );
  }
}
```

Ini adalah file paling penting untuk tugas Local Data & Persistence.

Karena di sini terdapat:

CREATE
READ
UPDATE
DELETE

dan semuanya menggunakan:

SharedPreferences

---

6. State Management

Kemudian:

lib/providers/booking_provider.dart

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../models/booking_model.dart';
import '../repositories/booking_local_repository.dart';

final bookingRepositoryProvider =
    Provider<BookingLocalRepository>((ref) {
  return BookingLocalRepository();
});

final bookingProvider =
    NotifierProvider<BookingNotifier, List<BookingModel>>(
  BookingNotifier.new,
```

```
);

class BookingNotifier
    extends Notifier<List<BookingModel>> {
  BookingLocalRepository get repository =>
      ref.read(bookingRepositoryProvider);

  @override
  List<BookingModel> build() {
    _loadBookings();
    return [];
  }

  Future<void> _loadBookings() async {
    state = await repository.getBookings();
  }

  Future<void> createBooking(
    BookingModel booking,
  ) async {
    await repository.createBooking(booking);
    await _loadBookings();
  }

  Future<void> updateBooking(
    BookingModel booking,
  ) async {
    await repository.updateBooking(booking);
    await _loadBookings();
  }

  Future<void> deleteBooking(
    String id,
  ) async {
    await repository.deleteBooking(id);
    await _loadBookings();
  }
}
```

Jadi alurnya:

UI
↓
BookingProvider
↓

BookingLocalRepository
↓
SharedPreferences

---

7. CRUD yang harus didemokan

CREATE

User mengisi:

Food : Nasi Ayam
Nama : Michael
Phone : 08123456789
Quantity : 2

Klik:

[BOOKING]

Data masuk ke:

SharedPreferences

READ

Saat aplikasi dibuka:

BookingProvider
↓
Repository
↓
SharedPreferences
↓
Booking List
↓
UI

Booking yang sebelumnya dibuat muncul kembali.

UPDATE

Misalnya:

Quantity: 2

diubah menjadi:

Quantity: 3

Kemudian:

[UPDATE]

Repository menjalankan:

updateBooking()

DELETE

User menekan:

[DELETE]

Kemudian:

deleteBooking(id);

Data dihapus dari local storage.

---

8. Persistence Test

Ini bagian **wajib** dan paling penting untuk demo.

Setelah CREATE:

Food Poison
↓
Booking
↓
Nasi Ayam - Michael - Qty 2

Kemudian **tutup aplikasi**.

Buka kembali.

Kalau muncul:

Nasi Ayam
Michael
Quantity: 2

berarti:

> **Persistence berhasil.**

Untuk Flutter Web, kamu juga bisa melakukan:

Create
↓
Refresh Browser
↓
Data masih ada

Karena `shared_preferences` memiliki implementasi storage yang sesuai platform.

---

9. Cara menjelaskan ke reviewer

Kamu bisa menggunakan penjelasan berikut:

Local Data & Persistence — Food Poison

Saya menerapkan Local Data & Persistence pada fitur **Booking Food** secara vertikal dari Frontend sampai Local Storage.

Struktur implementasinya adalah:

`BookingScreen → BookingProvider → BookingLocalRepository → SharedPreferences`

Data Model

Model data booking berada pada:

`lib/models/booking_model.dart`

Model ini menentukan data yang disimpan, yaitu ID booking, ID food, nama food, nama customer, nomor telepon, dan jumlah booking.

Local Storage

Implementasi local storage berada pada:

`lib/repositories/booking_local_repository.dart`

Repository menggunakan `SharedPreferences` untuk menyimpan data booking secara lokal pada device/browser.

Repository menyediakan empat operasi CRUD:

- `createBooking()` untuk membuat data baru.
- `getBookings()` untuk membaca data.
- `updateBooking()` untuk mengubah data.
- `deleteBooking()` untuk menghapus data.

Data model dikonversi menjadi JSON sebelum disimpan ke `SharedPreferences` dan dikonversi kembali menjadi `BookingModel` ketika dibaca.

State Management

State management berada pada:

`lib/providers/booking_provider.dart`

Provider memanggil repository untuk melakukan operasi CRUD dan kemudian memperbarui state yang ditampilkan oleh UI.

Frontend

Frontend berada pada:

`lib/screens/booking_screen.dart`

dan widget form berada pada:

`lib/widgets/booking_form.dart`

User melakukan interaksi melalui frontend, kemudian request diteruskan ke provider, repository, dan akhirnya disimpan pada local storage.

Persistence Test

Untuk membuktikan persistence, saya melakukan proses Create Booking kemudian menutup aplikasi dan menjalankannya kembali. Data booking yang sebelumnya dibuat tetap muncul.

Pada Flutter Web, saya juga melakukan refresh browser dan memeriksa kembali daftar booking. Data tetap tersedia sehingga membuktikan bahwa data tidak hanya tersimpan di state sementara aplikasi.

Vertical Feature

Satu fitur dikerjakan secara vertikal:

`Frontend → State Management → Repository → Local Storage`

Dengan demikian implementasi tidak hanya membuat UI, tetapi benar-benar menghubungkan fitur Booking Food sampai ke penyimpanan lokal.

---

10. Bukti demo yang paling bagus

Buat video pendek sekitar **1–2 menit**:

1. Buka Food Poison
   ↓
2. Masuk Booking
   ↓
3. CREATE booking
   ↓
4. Tunjukkan data
   ↓
5. UPDATE quantity
   ↓
6. Tunjukkan perubahan
   ↓
7. DELETE booking
   ↓
8. Buat booking lagi
   ↓
9. Tutup aplikasi / refresh browser
   ↓
10. Buka kembali

↓
11. Data masih ada

Screenshot kode yang paling penting:

```
lib/
├── models/
│   └── booking_model.dart            ← DATA MODEL
│
├── repositories/
│   └── booking_local_repository.dart   ← LOCAL STORAGE + CRUD
│
├── providers/
│   └── booking_provider.dart         ← STATE MANAGEMENT
│
└── screens/
    └── booking_screen.dart           ← FRONTEND
```

Kalimat kunci untuk reviewer:

> **"Local Data & Persistence saya berada di `booking_local_repository.dart`. Repository menggunakan SharedPreferences sebagai local storage dan menyediakan CRUD. Data didefinisikan oleh `BookingModel`, sedangkan `BookingProvider` menghubungkan repository dengan frontend. Saya menguji persistence dengan menutup aplikasi/refresh browser dan memastikan data booking tetap tersedia."**

Dengan pola ini, tugas **"satu fitur secara vertikal: FE → Local Storage"** terlihat jelas dan tidak sekadar membuat file storage yang tidak terhubung ke UI.